HACKATHON EXECUTION PLAYBOOK

Pollution Spotter

A four-member pipeline from photo to public report.

Build a live map of visual pollution reports, then prepare a complaint the user can review and send.

24-hour core demo · Optional improvement to 36 hours · 48 tracked tasks

THE DEMO JOURNEY

01 **PHOTO** — Capture + pin → 02 **VISION** — Visible clues → 03 **REVIEW** — Human confirms → 04 **MAP** — Report density → 05 **COMPLAINT** — Copy / download

START WITH THE TEAM, THEN FOLLOW YOUR TASK IDs

01 ALIGN
Pages 2–5
Scope, stack and shared contracts

02 BUILD
Pages 6–13
Twelve tasks for each member

03 SHIP
Pages 14–22
Integration, GitHub and demo

M1 Frontend M2 Backend M3 Vision M4 Map + demo

A planning pack, ready to put on GitHub. All tasks start unchecked. Replace member placeholders with your team's names; no repository or app has been created.

START HERE / SHARED AGREEMENT

What your four-person team will ship

Build a working Pollution Spotter demonstration in 24 hours, with optional improvement time through hour 36. This pack supplies a plan and GitHub checklists; application paths below describe files the team will create. The application, remote repository and GitHub Issues have not been created by this planning pack.

Use the pack in this order

Read these four shared pages together. Each member then follows their two role pages in task-ID order. Use the later reference pages when setting up GitHub, integrating, testing and rehearsing. H0 means the hackathon start; hour windows overlap across members and are not added together. Replace Member 1-4 and @member1-4 with your actual teammates.

Ownership and the main handoff

Owner	Builds	Hands to
M1 / frontend	Photo, location, review, complaint	M2: inputs; M4: refresh signal
M2 / backend	Contracts, auth, database, routes, deploy	Everyone: one stable interface
M3 / vision	Adapter, prompt, fixtures, evaluation	M2: validated visual suggestions
M4 / map + QA	Map, report density, tracker, demo	Everyone: integration evidence

The required demonstration is complete when

- A phone on HTTPS captures/selects a photo and lets the user confirm or correct its location without requiring GPS permission.
- A private draft shows visible-label suggestions or an honest fallback; the user explicitly confirms labels before publication.
- Publishing updates that draft once. The map refreshes immediately and a second visible browser discovers it through 10-second polling.
- Recent pollution report records form a clearly explained report-density hotspot. Real and labeled sample views keep separate counts.
- The complaint can be edited locally, copied and downloaded; ownership, error recovery and the deployed phone journey are checked.

Keep the scope finishable

> Use existing vision capabilities; do not train a model. This is a controlled hackathon demo. Automatic authority contact, accounts UI, chatbot, payments and production-scale operations are outside the required build. Visual labels and hotspots do not establish water safety or verified environmental danger. GitHub Issue checkboxes record completion; PDF and repository Markdown boxes do not synchronize with them.

SHARED DESIGN / DATA FLOW

One application, one visible journey

Use one Next.js repository so the frontend and server share TypeScript contracts. Agree on this flow in M1-01, M2-02, M3-02 and M4-02 before building separate features.

Beginner-friendly stack

Layer	Choice	Purpose
App + server	Next.js App Router, TypeScript, Tailwind	One project and shared types
Data + identity	Supabase Postgres, private Storage, anonymous auth	Private drafts without signup screens
Vision	@google/genai; available Flash-family vision model	Server-side image suggestions
Map	React Leaflet, Leaflet, OpenStreetMap	Clickable map with attribution
Hosting	Vercel; supported Node LTS	HTTPS deployment and phone access

End-to-end pipeline

1. Browser: obtain an anonymous Supabase session through M2's lib/supabase/browser.ts. Only the public URL/key belongs in this helper.
2. Input: select JPEG/PNG/WebP, reject sources over 8 MB, compress to at most 3 MB and 1600 px, then confirm a map pin.
3. Submit: POST multipart data with a retained client_request_id UUID and the anonymous JWT in the Authorization bearer header.
4. Reserve: the server verifies getUser(token), validates input and reserves one owner-scoped needs_review/unclear draft using the unique request key.
5. Store: decode with sharp, reject invalid or excessive images, strip metadata and save a normalized private image. Publication requires image_path to be ready.
6. Analyze: the first request invokes M3's adapter within a 12-second deadline. Valid output becomes draft/vision; failures retain manual_fallback and unclear.
7. Review: show the image, evidence and source. The owner edits labels/location as needed and explicitly confirms before publishing.
8. Publish: PATCH updates the same row and generates a safe complaint. Late AI writes cannot overwrite published, reviewed fields.
9. Display: GET exposes only published safe fields and signed photo URLs. Refresh the map, count report density, and offer editable complaint copy/download.

Make the boundaries visible

> M1 mounts M4's map through a client-only wrapper with SSR disabled. Gemini and service-role clients stay server-only. Every image result needs human confirmation; mock output forces the sample view. The map uses polling, not Supabase Realtime. OpenStreetMap attribution remains visible.

SHARED CONTRACT / FREEZE AT H2

Requests, records and retry behavior

M2 owns lib/contracts.ts and database/schema.sql. Use the same field names and validation on every branch. Success envelopes are {report} for POST/PATCH and {reports} for GET; errors use {error: {code, message, retryable}}.

API contract

Route	Input	Result
POST /api/reports	Multipart: image, latitude, longitude, client_request_id; optional note	Owner-only draft, source and review state
PATCH /api/reports/{id}	JSON: action=publish, labels, latitude, longitude	Same report ID, now published; safe complaint
GET /api/reports	Public read; no write token	Published safe fields and signed image URLs

Input and label rules

Writes require Authorization: Bearer <anonymous-session JWT>. Verify the token and owner on the server. Validate finite latitude [-90,90], longitude [-180,180], UUIDs and note length <=500. Ignore client owner_id, analysis_source and is_demo. Labels are algal_bloom, oil_sheen, garbage, no_visible_pollution or unclear. Pollution labels may co-occur; neutral labels must be exclusive. Require a nonempty, unique label set.

Report record

Fields	Rule
id; owner_id; client_request_id	UUIDs; UNIQUE(owner_id, client_request_id)
created_at; published_at	UTC timestamptz; published_at initially null
latitude; longitude; image_path	Coordinates; private path null until upload ready
labels; evidence; note	Label array; evidence <=240 chars; private optional note
status; analysis_source	draft|needs_review|published; vision|manual_fallback|mock
complaint_text; is_demo	Safe generated text; false except server-controlled samples

Exact public field allowlist

```
id, created_at, published_at, latitude, longitude, labels,
evidence, analysis_source, is_demo, image_url, complaint_text
```

Idempotency is part of the core flow

Keep the same UUID and input values on retry. Reserve needs_review/unclear first. A duplicate reuses the row; if needed, resume normalized upload at the same deterministic object key. Only the reservation-winning request may invoke AI; duplicates never start another model call. Return the current reviewable draft once its image exists. Otherwise return an actionable retry state. Publish requires a stored image; repeated publish preserves the ID and original published_at. AI updates must exclude published rows. In mock mode, reserve is_demo=true immediately and preserve it on every write, including an early retry/publication.

TEAM CLOCK / GATES BEFORE POLISH

The first 24 hours, then optional depth

All 48 role tasks belong to the required 24-hour plan. Work within a window can overlap between members. At each gate, demonstrate the actual path on the shared branch and record evidence instead of reporting percentage complete.

Required delivery schedule

Window	Parallel work	Exit evidence
H0-2	M2 scaffolds/deploys; M3 model smoke test; M1/M4 freeze interfaces	All can run; phone URL; exact model/mode; contracts
H2-6	M1 input; M2 draft/store; M3 adapter/fixtures; M4 map	H6: fixture-driven draft/review/map/complaint preview
H6-12	Connect live vision, human review, publish, map and complaint	H12: full live deployed journey; document any blocked gate
H12-15	Ownership, retries, edge cases and 20-image evaluation	Recorded failures, passing checks, assigned blockers
H15-18	Fix defects; improve phone layout; prepare sample seeds	Stable sample separation and repeatable recovery
H18-22	Phone + second-browser test; rehearse all four speakers	Core journey twice; timed demo; backup recording
H22-24	Freeze, verify release, close evidence-backed tasks	One final URL/commit; honest three-minute demo

Coordination rules that protect the deadline

- At H0, map names to M1-M4, choose one demo area and agree the deadline. Each member opens their role Issue and starts task 01.
- Every two hours, spend five minutes on: what now works, what blocks the next handoff, and who pairs to resolve it. Escalate a blocker after 20 minutes.
- M2 coordinates shared dependencies, lockfile, schema and environment changes. Use small branches and reviewed pull requests; keep the deployed main branch usable.
- If H6 or H12 fails, pause polish and pair across the failing interface. Cut animations, extra dashboards and elaborate styling before review, ownership or sample-label requirements.

Optional H24-36, only after required acceptance

Window	Choose a bounded improvement
H24-28	M3 expands unseen evaluation; M1 improves the clearest usability issue
H28-32	M2 improves diagnostics/recovery; M4 clarifies hotspot boundaries
H32-36	Retest changed paths, update evidence and rehearse the final release

A clear contingency is acceptable; concealed simulation is not

> H6 is a fixture-driven contract preview, not a completed backend. The publish route is built at H8-10. H12 requires the full live journey. If model access blocks it, record the live gate as unmet and demonstrate manual review or labeled mock/sample scenes as an explicit contingency.

M1 / TASKS 01–06 / BUILD

Frontend & reporting

Build a clear phone-friendly journey from photo and location to human review, publication and an editable complaint.

Owner: @member1 (replace) | **Reviewer:** Member 4 reviews usability; Member 2 reviews API integration.
Owns: app/page.tsx; components/report/*; components/complaint/*

- [ ] **M1-01 Agree on screens and API contracts** H0-1

  1. Read lib/contracts.ts with M2; agree on upload, analyzing, review, publishing, success and error states.
  2. Sketch these states for app/page.tsx; agree with M4 on selected-location and after-publish callbacks.
  Done: Team agrees on payloads, labels, mandatory human confirmation and map callbacks before coding.
  Handoff: M2 provides contracts/session helper; M4 confirms map props.

- [ ] **M1-02 Build the reporting page shell** H1-2

  1. In app/page.tsx and components/report/ReportWorkspace.tsx, create a phone-first page with labeled steps and visible progress.
  2. Keep browser APIs in the client workspace; reserve a client-only dynamic map slot for M4 with SSR disabled.
  Done: Page opens locally and on the initial HTTPS deployment without hydration or window errors.
  Handoff: M4 receives the map slot; M2 deploys the shared branch.

- [ ] **M1-03 Add camera, file selection and preview** H2-3

  1. Create components/report/PhotoInput.tsx with camera/file inputs, preview, replacement and clear supported-format guidance.
  2. Accept JPEG/PNG/WebP, reject sources over 8 MB, and resize/compress to at most 1600 px and 3 MB before submission; show a useful error if this fails.
  Done: Phone photo and file uploads preview correctly; unsupported and oversized files cannot proceed.
  Handoff: M2 receives the compressed image; server still validates and strips metadata.

- [ ] **M1-04 Let users confirm the location** H3-4

  1. In components/report/LocationStep.tsx, request browser location only after the user taps the location button.
  2. Connect M4's picker; allow tap/drag correction and manual pin selection when permission is denied or location is unavailable.
  Done: Confirmed valid coordinates are visible; denying GPS permission still allows completing a report.
  Handoff: Depends on M4-04; pass confirmed latitude/longitude to M2's draft route.

- [ ] **M1-05 Submit a private draft reliably** H4-6

  1. In ReportWorkspace.tsx, obtain M2's anonymous session and POST multipart image, coordinates, optional note and client_request_id with the bearer token.
  2. Disable repeat clicks; retain the same UUID and values for timeout retries. A new-report action creates a new UUID.
  Done: H6 mocked slice receives a draft; errors preserve inputs and never display false submission success.
  Handoff: M2 supplies idempotent draft route; M3's mock responses remain visibly sample data.

- [ ] **M1-06 Build mandatory human review** H6-8

  1. Create components/report/ReviewPanel.tsx showing the photo, visible-cue evidence, analysis source and editable labels/location.
  2. Allow multiple pollution labels; make unclear/no_visible_pollution exclusive. Require explicit confirmation before publishing; manual fallback starts as unclear.
  Done: User can correct AI output; unchecked confirmation blocks publication and no confidence or safety claim is shown.
  Handoff: M3 checks label/evidence wording; M2 validates the same rules server-side.

M1 / TASKS 07–12 / INTEGRATE & VERIFY

Frontend & reporting

Build a clear phone-friendly journey from photo and location to human review, publication and an editable complaint.

Owner: @member1 (replace) | **Reviewer:** Member 4 reviews usability; Member 2 reviews API integration.

- [ ] **M1-07 Publish the reviewed draft** H8-10

  1. Wire ReviewPanel.tsx to PATCH /api/reports/{id} with action=publish, edited labels/location and the bearer token.
  2. Show success only after the response; pass the published report to the workspace and trigger M4's immediate map refresh.

  Done: One reviewed draft becomes one published report; failed publication retains edits; mock reports enter only the sample view.

  Handoff: M2 updates the existing row; M4 verifies the new marker and count.

- [ ] **M1-08 Add editable complaint copy and download** H10-12

  1. Create components/complaint/ComplaintDraft.tsx using the server's safe complaint_text after publication.
  2. Allow local editing, Copy and Download .txt; explain that the user verifies the recipient and sends the complaint separately.

  Done: Both export actions work; private notes/contact details are absent from the public template and local edits stay local.

  Handoff: M2 supplies deterministic wording; M4 includes this screen in the H12 complete-flow gate.

- [ ] **M1-09 Make recovery paths understandable** H12-15

  1. Check report components for denied GPS, canceled selection, invalid images, draft failure, vision timeout and failed publication.
  2. Add concise recovery instructions, retained inputs and safe Retry/New report actions; distinguish AI fallback from failed draft creation.

  Done: Each failure has a usable next action; retries cannot silently create duplicate submissions.

  Handoff: M4 records failures; M2/M3 resolve backend or adapter problems.

- [ ] **M1-10 Check phone layout and accessibility** H15-18

  1. Check report/complaint components at 360 px width and with keyboard navigation; fix clipped controls and misplaced focus.
  2. Use text labels, readable contrast and status announcements; keep sample badges and the visual-suggestion explanation visible.

  Done: All required actions work without hover, color-only meaning or horizontal scrolling on a phone.

  Handoff: M4 reviews the complete phone journey; M2 merges the verified frontend changes.

- [ ] **M1-11 Run the deployed photo-to-complaint journey** H18-22

  1. On the deployed HTTPS URL, capture/select a real photo, confirm a pin, review labels, publish and copy/download the complaint.
  2. Repeat a denied-location or AI-fallback path; fix frontend defects and send screenshots/results to M4 for docs/demo.md.

  Done: M4 witnesses a successful phone flow and a recoverable failure; published data appears on a second browser.

  Handoff: M2 checks deployed API/session behavior; M4 collects final acceptance evidence.

- [ ] **M1-12 Freeze the frontend and hand off the demo** H22-24

  1. Merge reviewed frontend fixes; verify the final deployment and rehearse the photo, review and complaint steps with M4.
  2. Update the M1 GitHub Issue checkboxes with evidence links; record any remaining limitation instead of marking it complete.

  Done: M1's required tasks are verified, demo controls are stable, and M4 has usable screenshots plus a backup recording.

  Handoff: M2 freezes the release; M4 runs the judging demonstration.

M2 / TASKS 01–06 / BUILD

Backend & deployment

Turn a private photo draft into one reviewed, published report through secure, predictable server routes.

Owner: @member2 (replace) | **Reviewer:** Member 3 reviews image/model boundaries; Member 4 verifies deployed integration.
Owns: app/api/*; lib/server/*; lib/contracts.ts; lib/supabase/browser.ts (public anonymous auth only); database/*; root configuration

☐ M2-01 Create the shared development foundation — H0–1

1. Scaffold Next.js App Router with TypeScript and Tailwind; verify supported Node LTS and commit one lockfile.
2. Create Supabase/Vercel projects; deploy the starter; share repository and preview URL.
3. Create .env.example with names only; keep real .env.local ignored. Coordinate dependency changes through M2.

Done: All four members can run the starter; the Vercel HTTPS preview opens on a phone; no real secrets are committed.
Handoff: M1/M4 start UI work; give M3 server environment variable names.

☐ M2-02 Freeze request, response and ownership contracts — H1–2

1. Define report types, label rules and schemas in lib/contracts.ts; agree POST/PATCH/GET examples with all members.
2. Require a client_request_id UUID reused for retries; agree Bearer anonymous-session JWTs and stable error codes.
3. Record GEMINI_MODEL, VISION_MODE and server-only key names in .env.example; only the Supabase URL/public key may use NEXT_PUBLIC.

Done: All owners accept sample JSON and field names; no frontend component needs a service-role or Gemini key.
Handoff: M1 sends request IDs/tokens; M3 matches VisionResult; M4 consumes PublicReport.

☐ M2-03 Build the private database and photo store — H2–3

1. Add database/schema.sql for reports, UTC timestamps, status/source enums and UNIQUE(owner_id, client_request_id).
2. Create a private report-photos bucket with image MIME/size restrictions; enable anonymous sign-in.
3. Enable report/storage RLS denying direct browser access; server routes use the service key and explicit ownership checks.

Done: Schema applies cleanly; an anonymous browser client cannot read or write rows/photos directly; the server can store one test image.
Handoff: Share table fields and bucket name with M3; give M1 anonymous-session setup instructions.

☐ M2-04 Implement authenticated server route boundaries — H3–4

1. Create server-only helpers in lib/server; verify write tokens with getUser(token) and use the verified user ID.
2. Validate UUIDs, finite latitude/longitude ranges, note length ≤500 and allowed multipart fields; ignore client ownership/source/demo claims.
3. Return consistent JSON errors for missing authentication, invalid inputs, unavailable services and missing/foreign drafts.

Done: Missing/invalid tokens fail; two anonymous sessions cannot access each other's draft; errors give M1 an actionable retry or correction state.
Handoff: M1 wires error rendering; M4 records ownership test cases.

☐ M2-05 Create one recoverable private draft — H4–6

1. POST /api/reports in Node: decode JPEG/PNG/WebP with sharp; reject >8 MB or >40 megapixels. Reserve is_demo=true immediately in mock mode.
2. Reserve owner/request-ID row; store normalized ≤1600 px metadata-free photo, then image_path. M1 sends ≤3 MB.
3. Retries reuse/resume the same row; only INSERT winner invokes AI. Storage/DB errors remain retryable.

Done: One private draft survives retries; publish stays blocked until image_path exists; failed storage never reports success.
Handoff: M1 renders drafts; M3 gets bytes; M4 verifies.

☐ M2-06 Connect vision with a bounded fallback — H6–8

1. Call M3's server-only analyzeImage({bytes,mimeType}) within a 12-second application timeout; revalidate its result.
2. Save valid output as draft/vision; retain needs_review, unclear and manual_fallback on provider, schema or timeout failures.
3. Guard delayed updates against published rows. Preserve explicit mock source and demo status; never log keys, photo bytes or base64.

Done: Live, timeout, invalid-output and mock cases return truthful drafts; late model results cannot overwrite publication.
Handoff: M1 shows review/source states; M3 verifies provider failures with M2.

M2 / TASKS 07–12 / INTEGRATE & VERIFY

Backend & deployment

Turn a private photo draft into one reviewed, published report through secure, predictable server routes.

Owner: @member2 (replace) | **Reviewer:** Member 3 reviews image/model boundaries; Member 4 verifies deployed integration.

- [ ] **M2-07 Publish the reviewed report in place** H8–10

  1. PATCH /api/reports/{id}, action=publish: authenticate owner, require stored image, validate edited location/labels.
  2. Enforce neutral exclusivity; generate complaints from reviewed labels, coordinates and UTC time, excluding private notes/contact information.
  3. Update the same row; preserve first published_at on repeat confirmation. Preserve reserved is_demo=true; mock source also forces it.

  Done: Only owners publish; repeated confirmation creates no duplicate; reviewed fallback publishes; mock data cannot enter the real layer.

  Handoff: M1 unlocks Copy/Download; M4 refreshes the map with the returned published ID.

- [ ] **M2-08 Expose a safe public map response** H10–12

  1. Implement GET /api/reports with published-only selection and an explicit PublicReport field allowlist.
  2. Exclude owner IDs, raw image_path fields and raw notes; add short-lived signed photo URLs and retain source/demo labels.
  3. Bound query size; document ordering/retention; verify map refresh fetches fresh results after publication.

  Done: H12 live upload/review/publish/map/complaint works; public JSON excludes drafts; every route preserves real/sample distinctions.

  Handoff: M4 uses production-shaped data for polling, hotspots and image-expiry recovery.

- [ ] **M2-09 Test the boundaries that can break the demo** H12–15

  1. Check cross-owner reads/publishes, invalid coordinates, oversized/corrupt images, unsupported MIME and exclusive-label violations.
  2. Retry the same request and confirmation; simulate model timeout and a delayed model response after publication.
  3. Inspect public JSON, signed-link expiry and built client assets for private fields/secrets; document failures for immediate fixes.

  Done: M4 reproduces ownership/privacy/retry checks; failures have truthful UI responses; no service secret reaches the browser.

  Handoff: Send verified cases and remaining limitations to M4.

- [ ] **M2-10 Prepare controlled sample data and deployment** H15–18

  1. Create local-only database/seed.ts using server credentials; mark sample rows is_demo=true and keep photos licensed/consented.
  2. Set Vercel environment values; verify Node runtime, model ID, storage and anonymous-auth configuration.
  3. Confirm provider quota/host request limits; use a small controlled dataset and bounded image/model requests.

  Done: Sample hotspots appear only in sample view; clients cannot set is_demo; deployment completes one real submission.

  Handoff: M4 receives the exact seed/reset command and dataset scope; M3 verifies live quota.

- [ ] **M2-11 Verify the deployed phone workflow** H18–22

  1. Join M4's phone run over HTTPS: camera/file image, location correction, live analysis, review, publish and signed-photo display.
  2. Recheck compressed upload size against Vercel limits; fix actionable errors for offline/retry and expired anonymous sessions.
  3. Run repository build/type checks; confirm preview and final deployment use the intended VISION_MODE.

  Done: The final URL passes the core phone journey twice; quota/network fallback remains clearly labeled; build/type checks pass.

  Handoff: M4 freezes the demo URL; M1 receives final error/response behavior.

- [ ] **M2-12 Freeze the backend and hand over recovery steps** H22–24

  1. Merge reviewed changes; record deployed commit, environment names and setup/seed commands.
  2. Give M4 recovery steps for quota failure, manual fallback and deliberate mock/sample mode; mode changes must preserve labels.
  3. Check completed GitHub Issue tasks with evidence; leave optional hardening open.

  Done: M4 can recover the demo from written instructions; deployment/configuration are recorded; no core backend task remains unresolved.

  Handoff: M4 incorporates operations notes into docs/demo.md and the judging rehearsal.

M3 / TASKS 01–06 / BUILD

Vision & evaluation

Produce conservative visual suggestions with measurable limitations and an honest, reliable fallback.

Owner: @member3 (replace) | **Reviewer:** Member 2 reviews server integration and secret handling; Member 1 reviews understandable labels.

Owns: lib/vision/*; evaluation/*; tests/vision/*; vision prompt, fixtures and evaluation evidence

- [ ] **M3-01 Prove the chosen model works immediately** H0–1

1. In Google AI Studio, select a currently available Flash-family vision model; verify access instead of copying an old model ID.
2. Coordinate @google/genai installation with M2; run a server-side image request using an environment key.
3. Save the passing exact GEMINI_MODEL; record latency/quota errors without keys/images.

Done: One real image returns successfully, or the team explicitly activates and documents manual/mock contingency.

Handoff: M2 gets settings; M4 knows live availability.

- [ ] **M3-02 Agree the vision boundary and display language** H1–2

1. Agree analyzeImage({bytes:Buffer,mimeType:string}): Promise<VisionResult> with M2 and import shared types from lib/contracts.ts.
2. Use only algal_bloom, oil_sheen, garbage, no_visible_pollution and unclear; neutral labels exclude all other labels.
3. Return labels, evidence ≤240 characters and review_needed. Tell M1 every result still requires human confirmation.

Done: M1/M2 accept the sample payload and renderable fixture; types and label meanings agree before the review screen is built.

Handoff: M1 uses plain-language label captions; M2 can build the adapter call before live integration.

- [ ] **M3-03 Assemble an honest visual evaluation set** H2–3

1. Collect 20 consented/licensed images in evaluation/ with provenance; avoid faces, number plates and private metadata.
2. Assign four primary examples per label, including glare mistaken for oil, reflections/vegetation, blur and mixed visible pollution.
3. Record expected visible-label sets in evaluation/cases.csv; two teammates flag ambiguous examples without forcing certainty.

Done: All 20 cases have provenance, expected labels and ambiguity notes; expectations describe visible appearance only.

Handoff: M1/M4 receive a small clearly marked demo subset; preserve separate evaluation cases.

- [ ] **M3-04 Write a conservative prompt and output schema** H3–4

1. Create lib/vision/prompt.ts: inspect visible water-surface cues; choose unclear when the photo cannot support a label.
2. Prohibit toxin, chemistry, drinking-safety and confidence-percentage claims; require brief evidence tied to visible features.
3. Define structured JSON and strict validation: nonempty allowed labels, uniqueness, neutral exclusivity, evidence length and boolean review_needed.

Done: Valid single/multiple pollution labels pass; empty labels, mixed neutrals, invented enums and oversized evidence fail.

Handoff: M2 imports the validator; M1 confirms the wording is understandable and editable.

- [ ] **M3-05 Build the server-only live vision adapter** H4–6

1. Implement server-only lib/vision/analyze.ts using @google/genai, GEMINI_MODEL and normalized image bytes.
2. Request structured JSON and validate the returned data; throw typed errors for empty, invalid, blocked or failed responses.
3. Configure provider timeout/retries to fit M2's 12-second deadline; avoid uncontrolled automatic retries and log only safe diagnostics.

Done: A sample produces valid VisionResult; malformed/provider-failed responses produce typed failures; logs contain no keys/images.

Handoff: M2 imports the adapter for the draft route; coordinate dependency changes through M2.

- [ ] **M3-06 Supply explicit mocks for the H6 thin slice** H4–6

1. Provide deterministic fixtures for successful labels, unclear, malformed output, timeout and provider error in tests/vision/fixtures.
2. Implement deliberate VISION_MODE=mock behavior; require server metadata analysis_source=mock and is_demo=true for publication.
3. Run the H6 upload/review contract check with M1/M2/M4; verify manual_fallback differs visibly from successful live vision.

Done: The thin slice works without model access; all participants can identify live, manual fallback and mock results correctly.

Handoff: M4 receives repeatable failure fixtures; M2 preserves source/demo metadata outside VisionResult.

M3 / TASKS 07–12 / INTEGRATE & VERIFY

Vision & evaluation

Produce conservative visual suggestions with measurable limitations and an honest, reliable fallback.

Owner: @member3 (replace) | **Reviewer:** Member 2 reviews server integration and secret handling; Member 1 reviews understandable labels.

- [ ] **M3-07 Harden validation and failure handling** H6–9

  1. Add meaningful tests for wrong types/enums, duplicate labels, neutral conflicts, overly long evidence, missing fields and provider failures.
  2. Verify rejected output becomes needs_review/unclear/manual_fallback through M2's route instead of invented AI success.
  3. Check a delayed provider response cannot overwrite a published row; confirm mock metadata survives review/publish.

  Done: Boundary tests pass and each invalid provider case produces a reviewable truthful fallback; delayed output cannot change a confirmed report.

  Handoff: M2 owns route/update fixes; M1 owns clear fallback explanations.

- [ ] **M3-08 Complete the live H12 integration gate** H9–12

  1. Run deployed uploads for garbage, surface-green water, sheen-like appearance and an unclear/clean image.
  2. Confirm normalization preserves detail and model evidence stays brief and tied to visible features.
  3. Review/edit labels before publish; verify source metadata, complaint wording and map behavior for neutral results.

  Done: A live image completes the journey; neutral/unclear results never become hotspots; failures remain usable through review.

  Handoff: M4 records H12 acceptance; M1/M2 fix any integration issue before presentation polish.

- [ ] **M3-09 Measure model mistakes on the sample set** H12–15

  1. Run 20 annotated images with the frozen model/prompt; save expected/predicted labels, latency and failure/source in evaluation/results.csv.
  2. Count exact label-set matches, per-label misses and false positives; inspect clean/glare examples for overcalling.
  3. Record invalid outputs and timeouts separately; do not count manual/mock results as successful AI predictions.

  Done: All 20 cases have results/false-positive notes; counts include denominators and describe this small sample only.

  Handoff: M4 gets honest judging evidence; M1 gets examples that need clearer review guidance.

- [ ] **M3-10 Fix the clearest errors and verify the change** H15–18

  1. Make one focused prompt revision for recurring glare/vegetation/ambiguity errors; preserve original results.
  2. Rerun affected cases and five newly sourced unseen checks; record versions and changed outcomes separately.
  3. Document persistent errors and strengthen review/unclear guidance; never present selected examples as broad accuracy.

  Done: Final prompt is versioned; regressions/limits are recorded; false positives have mitigation or documented limitations.

  Handoff: M2 deploys the final prompt; M4 uses only verified claims in the demo script.

- [ ] **M3-11 Rehearse live and contingency demonstrations** H18–22

  1. Check model access/quota; run a real phone photo on the final deployment with M4.
  2. Rehearse unavailable-provider manual review and deliberate mock mode; mock output enters only the sample layer.
  3. Prepare three sourced demo photos; explain why visual labels cannot establish water safety.

  Done: Live/contingency paths are rehearsed; the presenter names the active mode and explains limits without unsupported claims.

  Handoff: M4 gets photos, provenance, expected behavior and a short vision explanation.

- [ ] **M3-12 Freeze vision and attach evidence to GitHub** H22–24

  1. Merge reviewed changes; record exact model ID, prompt version, test command and limitations in evaluation/README.md.
  2. Link schema-test output/evaluation counts in the assigned Issue; check completed items; leave optional improvements open.
  3. Help M4 capture a backup identifying sample/manual/mock content; avoid late model/prompt changes.

  Done: Frozen model/prompt matches deployment; teammates can open evidence links; fallback never appears as live AI success.

  Handoff: M4 incorporates evaluation and contingency evidence into docs/demo.md and the judging presentation.

M4 / TASKS 01–06 / BUILD

Map, integration & demo

Make published reports visible on a clear map, verify the end-to-end flow and keep GitHub progress and the demo accurate.

Owner: @member4 (replace) | **Reviewer:** Member 1 reviews map usability; Member 2 reviews data handling and integration.
Owns: components/map/*; lib/hotspots.ts; tests/e2e/*; docs/demo.md; planning trackers

- [ ] **M4-01 Set up the team's GitHub tracker** H0-1

  1. After the team creates its repository, copy the supplied role checklists into four GitHub Issues; replace @member1-4 placeholders with real teammates.
  2. Assign owners and add Todo, In progress, Blocked, Review and Done project statuses; link blockers and pull requests.
  Done: Every member has an assigned Issue and can tick its tasks; the team knows repository Markdown requires editing/committing.
  Handoff: All members use Issues as the task record; PDF boxes do not synchronize.

- [ ] **M4-02 Agree on map inputs and hotspot rules** H1-2

  1. Review public report fields with M2 and picker/refresh callbacks with M1; record agreements in docs/demo.md.
  2. Fix the demo center/reference latitude and agree on approximate 100 m projected bins, trailing 24-hour timestamps and separate real/sample counts.
  Done: Team agrees that three published pollution report records form a hotspot; records are not verified independent observations.
  Handoff: M2 supplies public report fixtures; M1 supplies the client-only map mount.

- [ ] **M4-03 Render the client-only Leaflet map** H2-3

  1. Build components/map/MapView.tsx with React Leaflet, Leaflet CSS and OpenStreetMap tiles; keep attribution visible.
  2. Show the chosen demo area, clear marker colors/labels and an honest empty state; default to real reports.
  Done: Map loads in M1's workspace without SSR errors, and attribution stays readable on a phone.
  Handoff: M1 imports MapView through the agreed client-only wrapper.

- [ ] **M4-04 Build the location picker** H3-4

  1. Create components/map/LocationPicker.tsx with tap-to-place and draggable pin behavior; emit validated latitude/longitude through the agreed callback.
  2. Accept M1's browser-location result for recentering and keep the selected pin visibly distinct from published markers.
  Done: M1 can select and correct a location without GPS; selecting a pin alone never creates a public report.
  Handoff: Unblocks M1-04; M1 stores the confirmed coordinates.

- [ ] **M4-05 Fetch and refresh published reports** H4-5

  1. In components/map/useReports.ts, use production-shaped fixtures until M2 provides GET /api/reports; fetch on load and after M1's publish callback.
  2. Poll every 10 seconds while visible, refresh on return, and clean up timers; show loading, error and last-successful-update states.
  Done: Production-shaped fixtures verify refresh and stale/error states; M4-08 verifies live second-browser updates after M2 delivers GET.
  Handoff: M2 guarantees published-only safe fields and fresh signed image URLs; describe this as polling.

- [ ] **M4-06 Calculate report-density hotspots** H5-6

  1. Implement the frozen projected-bin formula in lib/hotspots.ts; count each eligible published record once and split real/sample datasets.
  2. Check threshold, multilabel, time-boundary and neutral-label cases in tests/e2e/hotspots.spec.ts; show the H6 mocked map slice.
  Done: Three recent pollution records in one bin trigger a hotspot; unclear/no_visible_pollution, old or cross-layer records cannot inflate it.
  Handoff: M2 reviews timestamp/filter logic; M1 reviews the report-density wording.

M4 / TASKS 07–12 / INTEGRATE & VERIFY

Map, integration & demo

Make published reports visible on a clear map, verify the end-to-end flow and keep GitHub progress and the demo accurate.

Owner: @member4 (replace) | **Reviewer:** Member 1 reviews map usability; Member 2 reviews data handling and integration.

- [ ] **M4-07 Make sample data unmistakable** H6-8

  1. Add separate real and Sample demo views in MapView.tsx; label sample markers, popup details and hotspot counts clearly.
  2. Use only M2's server seed flow for demonstration fixtures; verify mock analysis reports appear exclusively in the sample view.
  Done: Switching views never combines counts; sample activity cannot be mistaken for real public reports.
  Handoff: M2 controls demo insertion; M3 provides labeled mock fixtures; M1 displays matching source badges.

- [ ] **M4-08 Pass the complete-flow integration gate** H8-12

  1. Exercise photo to draft to human review to publish to map to complaint; record the result in docs/demo.md.
  2. Verify drafts stay hidden, publication keeps one report ID, immediate refresh works, and a second browser receives the report through polling.
  Done: H12 deployed flow works with real vision. Record labeled manual/mock contingency if needed, but leave the live gate unmet.
  Handoff: Route blockers to M1/M2/M3 immediately; defer visual polish until this gate passes.

- [ ] **M4-09 Verify ownership and public-data boundaries** H12-15

  1. With M2, use two anonymous sessions to verify one owner cannot edit/publish the other's draft and unauthenticated writes fail.
  2. Check public JSON excludes owner IDs, raw image_path fields and private notes; confirm private storage cannot be listed/read directly.
  Done: Record passing checks in docs/demo.md and useful regressions in tests/e2e/report-flow.spec.ts; no credentials appear in evidence.
  Handoff: M2 fixes authorization/storage issues before the release gate.

- [ ] **M4-10 Test failures and phone map behavior** H15-18

  1. Check slow/failed vision, duplicate retries, denied location, empty results, network loss and signed-photo URL refresh; log defects in GitHub Issues.
  2. Test map controls/popups at phone width, attribution, sample badges and polling cleanup when hidden or unmounted.
  Done: Required failure paths recover without fabricated success or duplicate counts; unresolved demo blockers have owners.
  Handoff: M1 fixes UX, M2 fixes requests/data, M3 fixes adapter failures; M4 retests the fixes.

- [ ] **M4-11 Rehearse the map-fill demonstration** H18-22

  1. Write docs/demo.md with a three-minute script: photo, reviewed visual suggestion, new marker, report-density hotspot and copied complaint.
  2. Rehearse with all members; use a clearly labeled sample view for staged map filling and explain that refresh uses 10-second polling.
  Done: Everyone knows their speaking/action cue; live behavior and sample scenes are distinguishable, with a recorded backup available.
  Handoff: M1 operates reporting screens; M2 monitors deployment; M3 explains vision limits; M4 narrates the map.

- [ ] **M4-12 Close verified tasks and freeze the demo** H22-24

  1. Run final phone and second-browser checks on M2's release URL; save commit hash, screenshots, video and known limitations in docs/demo.md.
  2. Tick verified Issue tasks, link evidence and move reviewed work to Done; leave unfinished items open and rehearse the backup switch.
  Done: The judging URL, release version, checked task evidence and honest backup demo are accessible to all four teammates.
  Handoff: All members stop feature work; optional H24-36 improvements start only after acceptance passes.

GITHUB / CHECK WORK AS IT FINISHES

Turn the plan into four trackable Issues

M4 owns M4-01 and the tracker. The supplied files are ready to upload into your future repository; this pack has not published them or created remote Issues. GitHub Issues are the team's completion record.

Set up once

1. Create one GitHub repository, invite the four teammates with appropriate write access, and commit the scaffold plus the planning files. M2 manages the shared app foundation.
2. Follow docs/GITHUB_SETUP.md. Create four Issues by copying github-issues/member-1.md through member-4.md into their bodies; replace placeholder handles and assign each owner.
3. Optionally add the four Issues to a GitHub Project with Todo, In progress, Blocked and Done statuses; add Review only if useful. Use scope:optional for later enhancements.
4. Link GITHUB_TRACKER.md and PIPELINE.md from README.md. Keep the Issue URLs in the tracker so everyone can find the live checklists.

A task body preserves the stable IDs

```
- [ ] M1-05 Submit a private draft reliably
  Done: one private draft; retries keep the same ID.
  Evidence: PR URL + test result or short recording.
- [ ] M1-06 Build mandatory human review
```

Repeat for each task

- Move your Issue to In progress and name the active task in a comment. Use a small branch such as m1/report-upload; pull the latest main before starting.
- Implement only the owned paths, run the relevant check and open a pull request referencing the task ID. Use 'Refs #issue' when more tasks remain in the role Issue.
- Ask the named reviewer to verify the task's done criterion. Attach a screenshot, command result or short recording that demonstrates the actual behavior.
- After review and merge, tick that task in the Issue body. Move the role Issue to Done only when all 12 required tasks pass. Leave unfinished work visible.
- For blockers, add a comment naming the dependency, owner and next action; link the blocking Issue. Update status after the blocker is resolved.

What checking a box actually changes

> Authorized teammates can toggle task-list checkboxes in GitHub Issue bodies/comments. Checkboxes inside repository Markdown require editing and committing the file. Neither synchronizes with the PDF or with another checklist. Avoid 'Closes #issue' on an early pull request for a role Issue containing unfinished tasks. Never place keys, tokens or raw private photos in evidence.

VISION / SUGGEST, VALIDATE, CONFIRM

A conservative prompt and honest evidence

M3 follows M3-01 through M3-12. Choose a currently available Flash-family vision model in the team's Google AI Studio account, prove it with a real image at H1, and freeze that exact GEMINI_MODEL. No model ID or quota is assumed to remain available.

Starter prompt for lib/vision/prompt.ts

```
Inspect this pond/canal photo for visible surface cues only.
Choose one or more pollution labels when visibly supported:
algal_bloom, oil_sheen, garbage.
Use no_visible_pollution only when none is visibly supported.
Use unclear if image quality or ambiguity prevents a judgment.
Neutral labels must appear alone. Never invent an extra label.
Return JSON: labels (array), evidence (string), review_needed (boolean).
Evidence must be <=240 characters and describe visible features.
Do not claim chemistry, toxins, water safety or confidence percentages.
Set review_needed=true for ambiguity or uncertain visual evidence.
```

Adapter boundary and validation

M2 imports server-only analyzeImage({bytes, mimeType}): Promise<VisionResult>. The result contains labels, evidence and review_needed; source metadata is set by the server outside the model result. Use structured output and validate types, allowed unique labels, neutral exclusivity, nonempty arrays, evidence length and boolean values. Invalid, blocked, empty or timed-out results trigger truthful manual_fallback/unclear. review_needed=false never bypasses the user's confirmation step.

Small evaluation plan

Step	Evidence to save
Prepare 20 images	Four primary examples per label; licensed/consented provenance; mixed/ambiguous cases
Annotate together	Two teammates agree expected visible-label sets; retain uncertainty notes
Run frozen prompt	Expected/predicted sets, source, validity, latency and timeout/error in results.csv
Count mistakes	Exact set matches /20; per-label misses and false positives with denominators
Revise once	Preserve baseline; rerun affected cases plus five new unseen checks

Make the limitations measurable

- Include glare that resembles oil, reflections/vegetation, blur, ordinary debris and mixed visible pollution. Visible resemblance does not verify its chemical cause.
- Count provider failures and malformed outputs separately. Manual/mock results never count as successful AI predictions; failed cases remain in the 20-case overall denominator.
- Save prompt version, exact model ID and outcomes in evaluation/README.md. Present results as a small demonstration sample, without a broad accuracy claim.

VISION_MODE=mock is deliberate demonstration mode: preserve analysis_source=mock and force is_demo=true on publication. Provider failure in live mode is manual fallback, never a fabricated live prediction. Keep keys, image bytes and base64 out of logs.

MAP + COMPLAINT / EXPLAIN THE OUTPUT

Count reports, then draft a careful complaint

M4 owns lib/hotspots.ts (M4-02, M4-06); M2 owns the safe complaint template (M2-07); M1 owns local editing and export (M1-08). Both outputs use the owner's reviewed labels.

Freeze one approximate local grid

At H0 choose one demo area and record its center latitude as phi0 in degrees. Keep phi0 fixed for all records and both layers; it is a projection reference, never a substitute for the user's location. Use this equirectangular approximation for a small local area away from the poles/dateline. Cell boundaries are approximate and can split nearby reports.

Exact formula and eligibility

```
R = 6371000 meters; s = 100 meters
x = R * (longitude * pi / 180) * cos(phi0 * pi / 180)
y = R * (latitude * pi / 180)
cell = (floor(x / s), floor(y / s))
window: now_UTC - 24 hours <= published_at <= now_UTC
pollution = {algal_bloom, oil_sheen, garbage}
eligible: published AND labels intersect pollution
hotspot: at least 3 distinct report IDs in the same layer/cell
```

Display rules

- Count a multilabel report once. Exclude unclear/no_visible_pollution from hotspots; neutral published reports may keep neutral markers. Separate is_demo=true and false before counting.
- Label the result 'Report density: 3 reports in about 100 m, last 24 h.' Distinct report IDs do not prove distinct observers or independent environmental events.
- Default to the real view; badge sample markers/popups/counts. Poll every 10 seconds while visible, refresh immediately after publish, and show last successful update or stale-data errors.
- For the controlled demo, return at most the latest 200 published reports from the last 24 hours. At that cap, label the limited dataset; density covers returned records and can undercount.

Safe editable complaint template

```
Subject: Request to inspect a reported water-surface concern
Location: [reviewed latitude, longitude]
Reported at: [published_at in UTC]
Observed appearance: [reviewed plain-language pollution labels]
Please inspect this location and advise on appropriate next steps.
This photo report is a visual observation, not a laboratory finding.
```

For unclear or no_visible_pollution, use neutral 'Request for inspection / observation' wording and do not assert a pollutant. Generate public text only from reviewed safe fields; omit private notes/contact details. Local edits need not be saved publicly. Copy/download only: the user verifies the authority and sends the complaint separately.

INTEGRATION / PROVE THE IMPORTANT BOUNDARIES

A practical acceptance and QA matrix

M4 coordinates M4-08 through M4-10 with M2-09 and M3-07. Put useful flow checks in tests/e2e/*, vision boundary checks in tests/vision/*, and concise manual evidence in docs/demo.md. Check behavior and failure recovery, not styling details.

Required checks before the release

Case	Expected result	Lead
Real phone photo	Preview, valid pin, draft, reviewed publish, map and complaint	M1 + M4
GPS denied	Manual pin still completes the flow	M1
Corrupt / unsupported / oversized image	Actionable rejection; no false success	M2
Timeout / malformed AI JSON	unclear + manual_fallback; review remains possible	M3 + M2
Retry same submission	Same owner/request key returns one report; no second AI call	M2
Publish twice / late AI	Same row and first published_at; reviewed fields unchanged	M2
Owner A vs owner B / no token	Foreign draft/write denied; unauthenticated writes fail	M2 + M4
Read public JSON / private storage	No drafts, owner IDs, raw notes or raw image_path; direct anon access denied	M2 + M4
Hotspot threshold + boundary	2 records do not qualify; 3 do; old/future/neutral excluded	M4
Multilabel + mock samples	One count per ID; real/sample never combined	M4
Polling / offline / photo expiry	Refresh/cleanup correct; stale state shown; signed URL renewed	M4
Complaint + mobile controls	Copy/download works; safe wording; keyboard/phone usable	M1

Run the tests at the right gates

- H6: M1-05, M2-05, M3-06 and M4-06 use fixtures for a draft/review/map/complaint contract preview. Resolve interface mismatches; full backend publication is scheduled later.
- H12: complete the full live deployed real-image journey. If access blocks live vision, record that gate as unmet and disclose the fallback demonstration.
- H18-22: run the core flow twice on HTTPS with a phone and a second visible browser. Check the final deployment, not only localhost.
- Before merging: run the changed area's meaningful tests plus the repository build/type checks. Rerun failing or changed paths after fixes.

Attach evidence to the task

> Record task ID, commit/deployed URL, input scenario, expected versus actual behavior, and a pass/fail result. Link a screenshot or short recording when useful. Keep tokens, service credentials and private input out of evidence. A test passes only when its expected behavior is observed; an untested box stays open.

SETUP / ACCOUNTS, ENVIRONMENT, COST

Make external dependencies work at hour one

M2 owns project setup (M2-01 through M2-03); M3 owns model access (M3-01). Use team-accessible accounts and record who can recover each service. Verify current limits in the actual accounts before committing the demo to them. Copyable commands and scaffold-merging steps: docs/APP_SETUP.md. Start in a new sibling folder; preserve the clone's planning files and .git.

Setup order

Step	Concrete action	Owner
1 / workstation	Install supported Node LTS/Git; scaffold Next.js with TS, Tailwind, App Router	M2
2 / packages	Coordinate @supabase/supabase-js, @google/genai, sharp, zod, react-leaflet, leaflet	M2
3 / repository	Commit one lockfile; all clone, install and run the same starter	All
4 / Supabase	Create project; enable anonymous auth; apply schema/RLS; private report-photos bucket	M2
5 / vision	Use Google AI Studio account; run one server-side image/model request	M3
6 / Vercel	Connect repository; set environments; deploy starter and open HTTPS on phone	M2 + M4

Names-only .env.example; real values stay private

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SERVICE_ROLE_KEY=
GEMINI_API_KEY=
GEMINI_MODEL=
VISION_MODE=live
```

Who reads which values

M2 creates lib/supabase/browser.ts using only the first two public values; a project browser publishable key or supported legacy anon key can fill the public-key setting. Server-only clients read service-role and Gemini secrets. Put real local values in ignored .env.local and hosting values in Vercel settings. Keep .env.example empty of credentials. Fix the map reference latitude in shared configuration and record it in docs/demo.md.

Hour-one access and cost gate

- Save the exact passing model ID, SDK version, request latency and any quota restriction. Confirm image input works; a text-only model response is insufficient.
- Check account billing/quota and hosting request/duration limits. Agree a small team spending ceiling and one owner; do not assume a free tier or automatically upgrade a plan.
- Budget approximately 20 baseline cases, focused reruns, five unseen checks and rehearsals; watch actual usage. Compress images and avoid repeated automatic provider retries.
- If live access is unavailable, record it at H1 and continue integration with explicitly labeled mocks/manual fallback while M3 investigates access.

> M2 owns dependency and environment changes to avoid conflicting lockfiles. Do not put any service-role or Gemini key in NEXT_PUBLIC variables, browser code, screenshots, Issue comments or a committed environment file.

DEPLOYMENT / PRIVATE DRAFTS, PUBLIC REPORTS

Ship one reliable HTTPS demonstration

M2 leads M2-10 through M2-12 with M4's verification. The deployment must preserve the same ownership, source labels and retry behavior as local development. Keep the final judging URL and deployed commit together in docs/demo.md.

Deployment sequence

1. Merge reviewed changes, install from the committed lockfile, and run the repository's build/type checks plus relevant tests. Resolve failures before declaring the release ready.
2. Apply database/schema.sql to the intended Supabase project; verify RLS and the private report-photos bucket. Enable anonymous sign-in and confirm the deployed client can obtain a session.
3. Set the same environment names on Vercel for the intended deployment. Set the tested GEMINI_MODEL and deliberate VISION_MODE, then redeploy after changes.
4. Run upload/model routes in the Node runtime for sharp and Buffer. Allow storage work plus the 12-second model deadline within the host's verified execution limit.
5. Test compressed images under the host request-body limit; keep the client image <=3 MB with room for multipart overhead. The server still rejects invalid images, >8 MB inputs and >40 megapixels.
6. On the final HTTPS URL, complete the phone journey twice and verify immediate/polled map updates from a second browser. Save the final commit and release evidence.

Security and publication boundaries

Boundary	Required behavior
Write identity	getUser(token); derive owner server-side; reject foreign drafts
Direct database/storage	RLS denies public/anon access; server clients remain secret
Public reports	Explicit allowlist; published only; no raw notes, owner IDs or separate image_path field
Public photos	Private bucket; signed URLs expose safe reports/{reportId}.jpg path and temporary read
Samples	Server-controlled samples; mock mode reserves is_demo=true immediately and preserves it
Logs	Safe error codes/latency; no keys, JWTs, image bytes or base64

Keep the public view current

Disable unintended caching so publication appears immediately. GET returns the latest 200 published reports within 24 hours, ordered by published_at descending; label the cap if reached. Signed URLs reveal the bucket and safe object path, never an owner ID; use reports/{reportId}.jpg and refresh URLs when needed. Anonymous identity belongs to the browser session; another browser or a lost session does not automatically recover its drafts.

> Publication makes the approved photo and location viewable through the application. Explain that at review. This is a controlled hackathon service; production moderation, abuse prevention, recovery, deletion and operational monitoring remain future work.

RECOVERY / KEEP FAILURE STATES TRUTHFUL

What to do when a dependency fails

M1 preserves understandable UI states, M2 preserves data correctness, M3 handles model boundaries, and M4 rehearses recovery. Do not change the demo's story to hide a failure; use a known alternate path and name it.

Recovery playbook

Failure	Required response	Owner
Camera/format unsupported	Offer file selection; request JPEG/PNG/WebP; retain usable inputs	M1
GPS denied/unavailable	Let the user tap/drag a map pin and confirm coordinates	M1 + M4
Image invalid/too large	Explain rejection; allow replacement/compression; no successful draft message	M1 + M2
DB/storage fails after reservation	Return retryable error; same UUID resumes same row/object; block publish without image	M2
Browser request times out	Retry same UUID and values; use existing row; never launch a duplicate AI call	M1 + M2
Model quota/timeout/invalid output	needs_review + unclear + manual_fallback; user reviews labels before publish	M3 + M2
Late model result	Guard update against published rows; preserve user-reviewed fields	M2
Session missing/expired	Refresh existing session if possible; explain lost ownership before starting a new report	M2 + M1
Map network/tiles fail	Show error/last update; retain stale reports with a label; use recording if necessary	M4
Signed photo URL expires	Refetch reports for fresh URLs; show a placeholder while recovering	M4 + M2
Live model unavailable at judging	Use disclosed manual review or deliberate mock/sample mode; disclose active source	M3 + M4

A ready draft is different from a pending reservation

A reserved row may have image_path=null while upload is pending. It must not become a public report or be shown as successfully ready for review. A retry may safely resume normalized storage at the same deterministic key; only the request that won the reservation invokes the model. Once the image exists, a duplicate can return the current draft without another provider call. If the first request never completes analysis, manual review is still a truthful path.

Prepare the switch before the demo

> Record the exact sample seed/reset command and deployment-mode change steps in docs/demo.md. Changing VISION_MODE may require redeployment; it is not a browser toggle. Keep a short recording and screenshots locally available. On failure, say which part is unavailable and demonstrate only what the current path actually does.

JUDGING / EXACT THREE-MINUTE RUN

Make the map-fill moment clear and credible

M4 owns the script in docs/demo.md (M4-11); all four members rehearse it. Prepare the phone, final HTTPS URL, second browser, a consented photo, labeled sample fixtures and a local backup recording before the clock starts.

180-second script

Time	Speaker/action	Show or say
0:00-0:20	M4 / introduce	A resident sees a pond/canal concern. We turn a photo into a reviewed report, a map and a complaint draft.
0:20-0:45	M1 / photo + pin	Capture/select the photo; confirm location. Show that GPS can be corrected or replaced with a manual pin.
0:45-1:10	M3 speaks; M1 reviews	Show source/evidence, edit if needed, then confirm. Explain that the model suggests visible appearance and does not test water safety.
1:10-1:30	M1 / publish	Publish once. Show immediate map refresh and the second browser updating through the visible 10-second polling cycle.
1:30-1:55	M4 / map fill	Switch to clearly labeled Sample demo. Start with two staged records; M2 adds the third via the prepared seed step; show the hotspot.
1:55-2:20	M1 / complaint	Open the generated text, make one local edit and copy/download. The user verifies the recipient and sends it separately.
2:20-2:40	M2 / engineering	One Next.js app, private drafts, owner-checked publication, idempotent retries and separate sample data. Show the deployed URL.
2:40-3:00	M3 + M4 / evidence	Give recorded sample-evaluation counts, one known confusion and the next improvement. Explain that hotspots measure report density.

Rehearse these mechanics

- Freeze a three-record sample group in one grid cell within 24 hours. Use licensed/consented photos; mark each is_demo=true and keep real/sample counts separate.
- M2 documents and runs the local-only baseline/third-record seed steps; the browser cannot insert arbitrary demo reports. Allow one polling cycle for the reveal.
- Use your actual evaluation results, never placeholder accuracy. If live analysis falls back, name manual fallback and still demonstrate the human-review step.

Backup without pretending it is live

> If a step stalls, state the failing dependency and switch to the rehearsed manual path or a clearly labeled recording. Continue through the remaining script using screenshots/video; identify recorded and sample content. Keep the backup locally available so network loss does not stop the explanation.

RELEASE / DONE MEANS DEMONSTRATED

Final readiness and each member's next step

At H22-24, each owner reviews their role Issue with the named reviewer. M4 checks shared acceptance, M2 records the deployed commit, and the team stops feature work. Optional improvements remain separate from required completion.

Final readiness checklist

- [ ] M1-11 / M2-11: the final HTTPS phone journey passes twice, including a real photo, corrected pin, confirmed labels, publish and complaint export.
- [ ] M2-09 / M3-07: ownership, private storage, bad inputs, retries, provider failure and late-AI protections have evidence.
- [ ] M3-09 / M3-12: exact model and prompt version are recorded; sample evaluation includes denominators and failures; limitations match the presentation.
- [ ] M4-06 / M4-07: hotspot formula and trailing-24-hour window are correct; sample/real separation is visible; counts make no independence or danger claim.
- [ ] M4-11: all four members rehearse the three-minute script, the third-sample reveal and a disclosed backup switch.
- [ ] M1-12 / M2-12 / M3-12 / M4-12: reviewed required tasks are checked with evidence; final URL, commit, seed/recovery steps and backups are shared.

Beginner learning path: learn only the next needed skill

Member	Learn in this order	Official starting point
M1	React state -> file inputs -> fetch/FormData -> review UX	nextjs.org/learn
M2	Route handlers -> Supabase auth/storage -> RLS -> deployment	supabase.com/docs; nextjs.org/docs
M3	Image request -> structured JSON -> validation -> evaluation	ai.google.dev/gemini-api/docs
M4	Leaflet map -> coordinates/bins -> refresh -> integration checks	react-leaflet.js.org; docs.github.com

What to do immediately

Each member opens task 01, names the output they will hand off, and starts within their owned paths. M2 creates the shared starter; M3 proves model access while M1 sketches the flow and M4 prepares tracking. Do not wait for the full app before testing interfaces: use the agreed sample responses at H6, then replace them with the real path and disclose the active source.

After the hackathon

> Expand unseen visual evaluation, improve usability from observed failures, and validate complaint routing with intended users. Before opening a public service, design moderation, abuse controls, account recovery and data deletion. Reliable water-safety or pollutant claims require appropriate measurements and domain review beyond this visual-reporting prototype.